

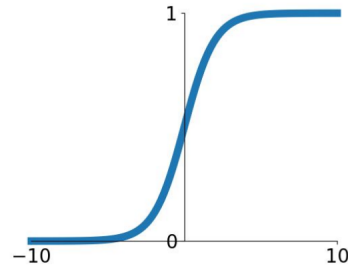
神经网络训练

选择Activation Functions

Activation Functions

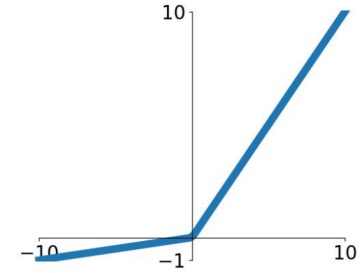
Sigmoid

$$\sigma(x) = \frac{1}{1+e^{-x}}$$



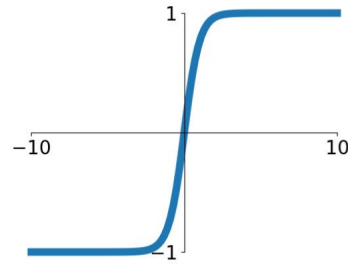
Leaky ReLU

$$\max(0.1x, x)$$



tanh

$$\tanh(x)$$

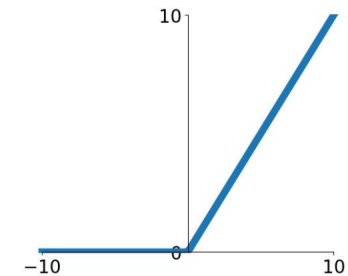


Maxout

$$\max(w_1^T x + b_1, w_2^T x + b_2)$$

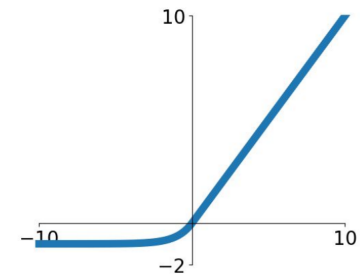
ReLU

$$\max(0, x)$$



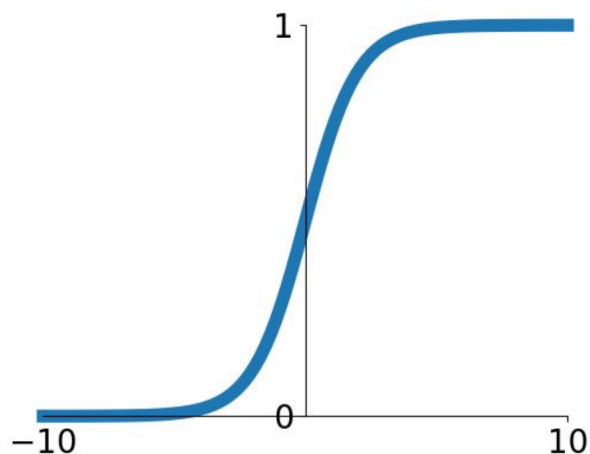
ELU

$$\begin{cases} x & x \geq 0 \\ \alpha(e^x - 1) & x < 0 \end{cases}$$



Sigmoid神经元

Activation Functions



Sigmoid

$$\sigma(x) = 1/(1 + e^{-x})$$

- Squashes numbers to range [0,1]
- Historically popular since they have nice interpretation as a saturating “firing rate” of a neuron

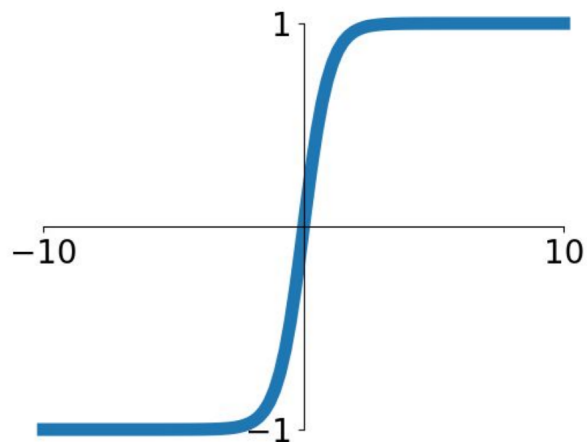
3 problems:

1. Saturated neurons “kill” the gradients
2. Sigmoid outputs are not zero-centered
3. $\exp()$ is a bit compute expensive

可解释性好：输出值可解释为神经元的激活概率(firing rate)

Tanh神经元

Activation Functions



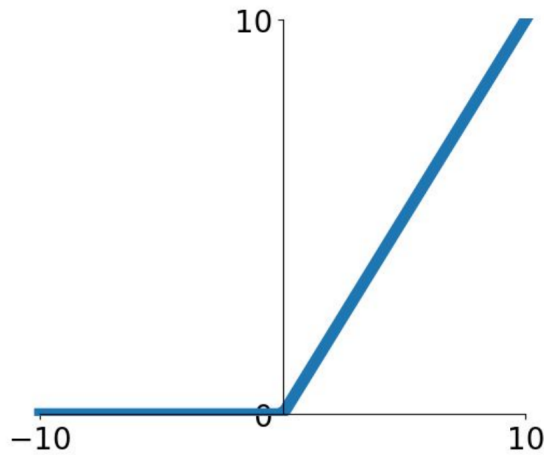
$\tanh(x)$

- Squashes numbers to range $[-1,1]$
- zero centered (nice)
- still kills gradients when saturated :(

[LeCun et al., 1991]

Relu神经元

Activation Functions



ReLU
(Rectified Linear Unit)

- Computes $f(x) = \max(0, x)$
- Does not saturate (in +region)
- Very computationally efficient
- Converges much faster than sigmoid/tanh in practice (e.g. 6x)
- Not zero-centered output
- An annoyance: dead ReLU
will never activate
=> never update

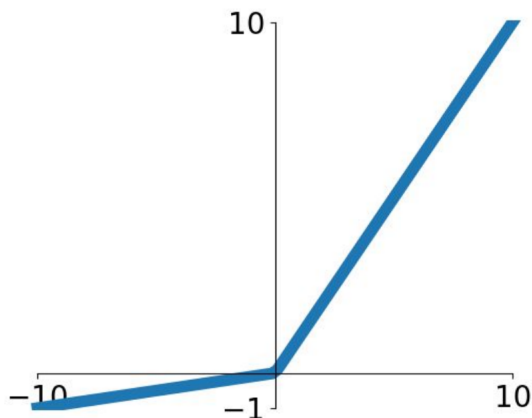
hint: what is the gradient when $x < 0$?

Leaky Relu神经元

Activation Functions

[Ivashka et al., 2015]

[He et al., 2015]



Leaky ReLU

$$f(x) = \max(0.01x, x)$$

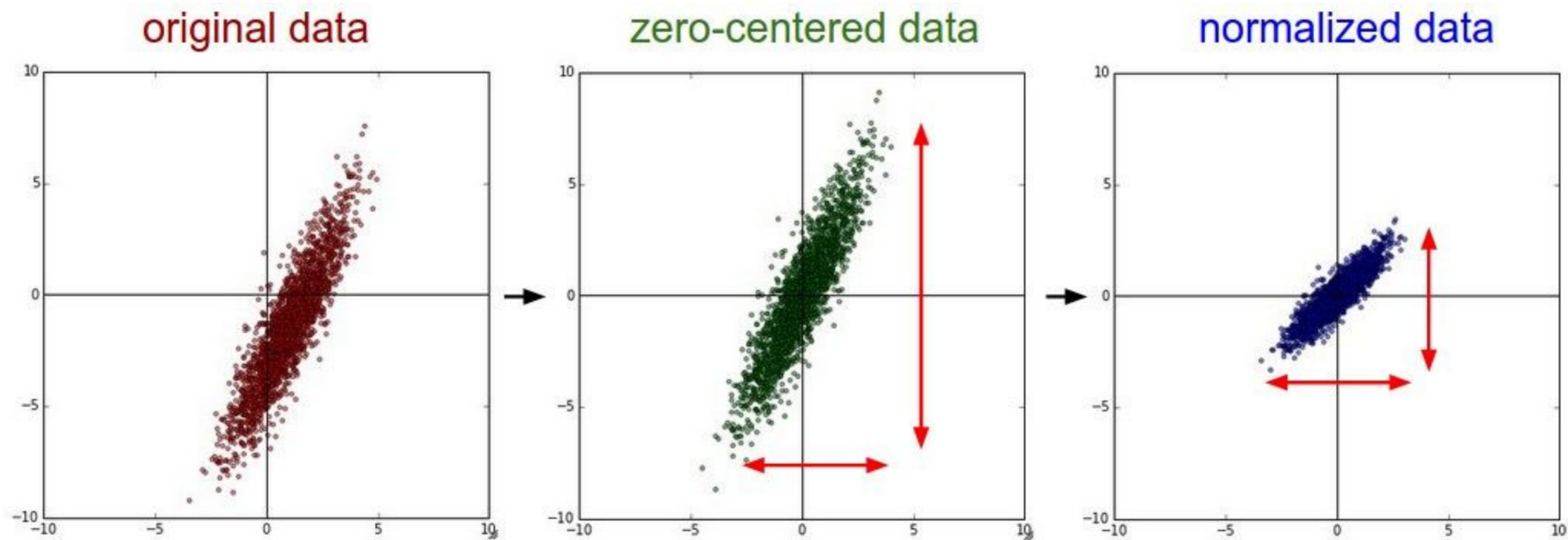
- Does not saturate
- Computationally efficient
- Converges much faster than sigmoid/tanh in practice! (e.g. 6x)
- **will not “die”.**

Parametric Rectifier (PReLU)

$$f(x) = \max(\alpha x, x)$$

backprop into α
(parameter)

Data Preprocessing: Data Normalization



```
X -= np.mean(X, axis = 0)
```

```
X /= np.std(X, axis = 0)
```

$$\mu_n = \frac{1}{m} \sum_{i=0}^m x_{ni} - \text{mean}$$

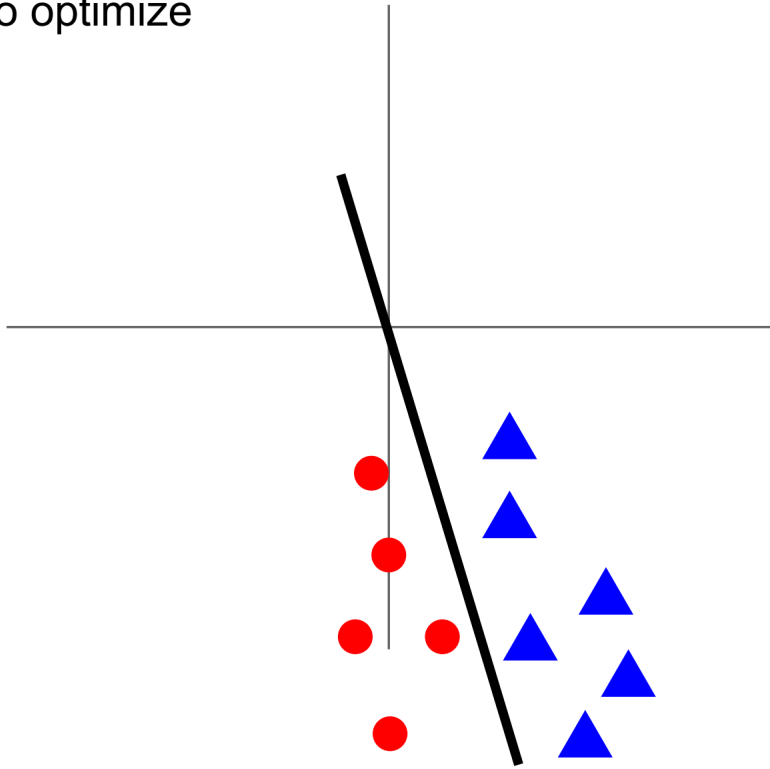
针对单一数据样本做标准化

$$\sigma_n = \sqrt{\frac{1}{m} \sum_{i=0}^m (x_{ni} - \mu_n)^2} - \text{std}$$

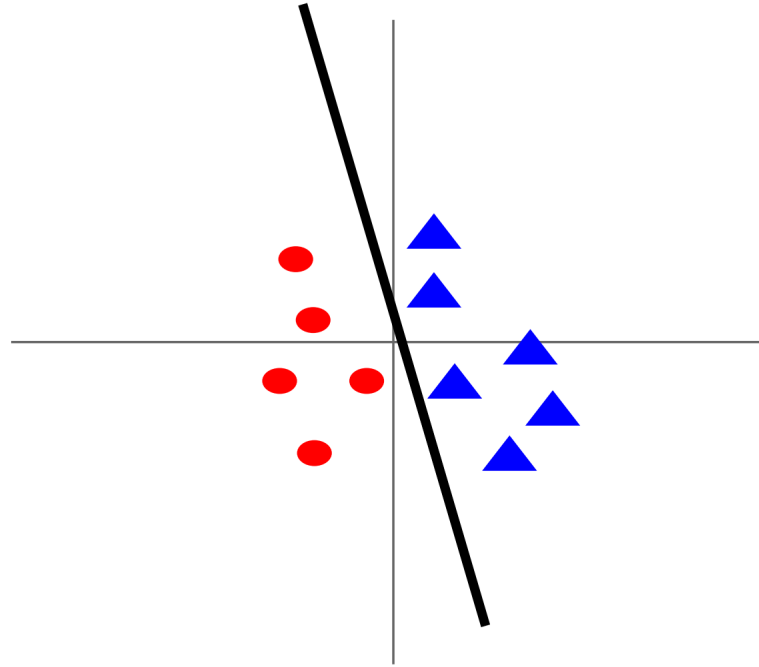
$$\hat{x}_n = \frac{x_n - \mu_n}{\sigma_n} - \text{normalized input}$$

Data Preprocessing: Data Normalization

Before normalization: classification loss very sensitive to changes in weight matrix; hard to optimize



After normalization: less sensitive to small changes in weights; easier to optimize



权重初始化

Xavier 初始化:

Glorot 和 Yoshua Bengio 在 2010 年提出的, 原文为 [Understanding the difficulty of training deep feedforward neural networks](#)。该初始化方法旨在保持激活函数的方差在前向传播和反向传播过程中大致相同, 从而避免梯度消失或梯度爆炸的问题。主要针对Tanh神经元。

1. 均匀分布 (Xavier Uniform) :

权重 w 服从

$$W \sim U[-\sqrt{\frac{6}{n_i + n_{i+1}}}, \sqrt{\frac{6}{n_i + n_{i+1}}}]$$

n_i : 第 i 层神经元数目, n_{i+1} : 第 $i+1$ 层神经元数目

2.正态分布 (Xavier Normal) : 权重 w 服从 $W \sim N[0, \frac{2}{n_i + n_{i+1}}]$

Xavier初始化的缺点: Xavier的推导过程是基于激活函数是线性的, 以及激活值关于0对称, Xavier初始化不适用于sigmoid函数和ReLU函数。

权重初始化

- **kaiming**初始化（针对Relu函数）

1 Kaiming Initialization

Consider the neural network with ReLU activation:

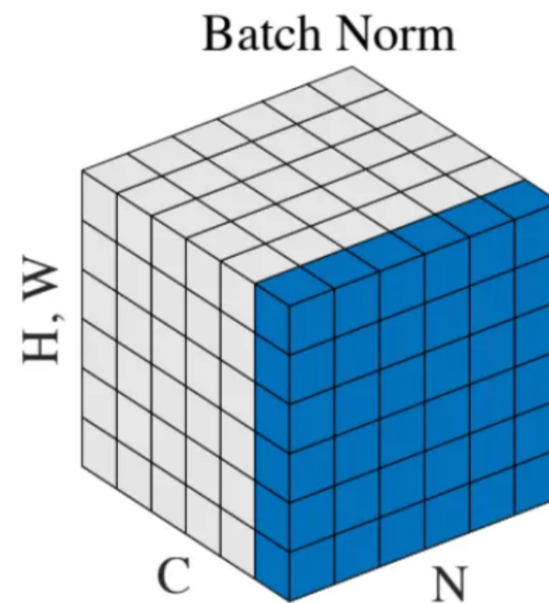
$$f(x, W^{(0)}, \dots, W^{(H-1)}, b^{(0)}, \dots, b^{(H-1)}) = W^{(H-1)} \sigma(W^{(H-2)} \dots \sigma(W^{(0)}x + b^{(0)}) \dots + b^{(H-2)}) + b^{(H-1)}$$

where $x \in \mathbb{R}^{d_0}$, $b^{(h)} \in \mathbb{R}^{d_{h+1}}$ and $W^{(h)} \in \mathbb{R}^{d_{h+1} \times d_h}$. Let $x^{(0)} = x$, and for $h \in \{0, \dots, H-1\}$, $z^{(h)} = W^{(h)}x^{(h)} + b^{(h)} \in \mathbb{R}^{d_{h+1}}$ and $x^{(h+1)} = \sigma(z^{(h)})$.

Kaiming Initialization [1] For any $h \in \{0, \dots, H-1\}$, initialize $b^{(h)} = 0$ and $W_{ij}^{(h)} \sim \mathcal{N}(0, \frac{2}{d_h})$ for $i \in [d_{h+1}]$ and $j \in [d_h]$.

Batch Normalization

- 2015 年，Sergey Ioffe 和 Christian Szegedy 采用BN解决内部协变量偏移问题。可简单理解为：输入层数据分布由于权重更新而不断变化。导致下一层数据分布总是需要不断适应新的分布。这会导致收敛速度变慢和训练不稳定。
- BN用来控制和优化每层产生的数据分布。该过程与输入标准化相同，但添加了两个可学习参数， γ 和 β 。

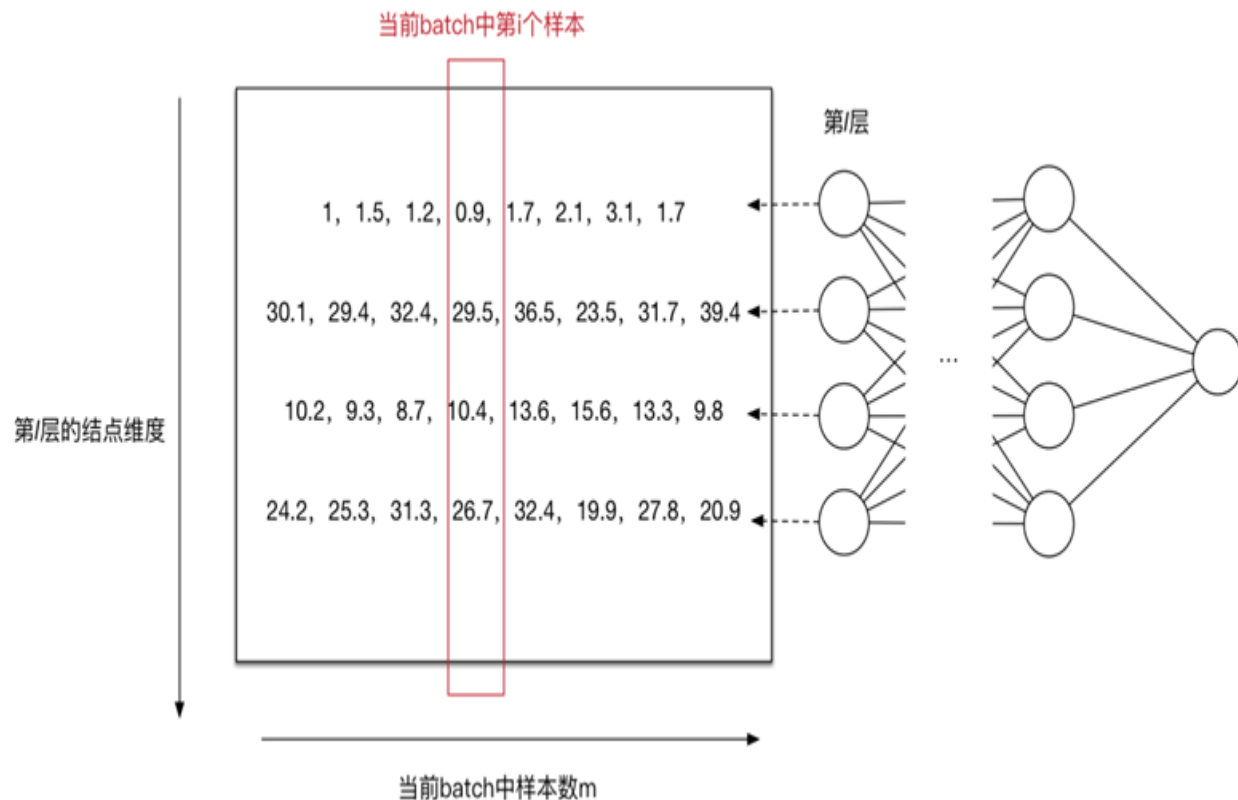


Batch Normalization

- 举例：对第一层做BN

$$\mu_j = \frac{1}{N} \sum_{i=1}^N x_{i,j} \quad \text{Per-channel mean, shape is D}$$
$$\sigma_j^2 = \frac{1}{N} \sum_{i=1}^N (x_{i,j} - \mu_j)^2 \quad \text{Per-channel var, shape is D}$$

$$\hat{x}_{i,j} = \frac{x_{i,j} - \mu_j}{\sqrt{\sigma_j^2 + \epsilon}}$$



对于第一个神经元，我们求得 $\mu_1 = 1.65$, $\sigma_1^2 = 0.44$ (其中 $\epsilon = 10^{-8}$) , 此时我们利用 μ_1, σ_1^2 对第一行数据 (第一个维度) 进行normalization得到新的值 $[-0.98, -0.23, -0.68, -1.13, 0.08, 0.68, 2.19, 0.08]$ 。同理我们可以计算出其他输入维度归一化后的值。

Batch Normalization

如同上面提到的，Normalization操作我们虽然缓解了ICS问题，让每一层网络的输入数据分布都变得稳定，但却导致了数据表达能力的缺失。也就是我们通过变换操作改变了原有数据的信息表达（representation ability of the network），使得底层网络学习到的参数信息丢失。另一方面，通过让每一层的输入分布均值为0，方差为1，会使得输入在经过sigmoid或tanh激活函数时，容易陷入非线性激活函数的线性区域。

因此，BN又引入了两个可学习（learnable）的参数 γ 与 β 。这两个参数的引入是为了恢复数据本身的表达能力，对规范化后的数据进行线性变换，即 $y_{i,j} = \gamma_j \hat{x}_{i,j} + \beta_j$ 。

Batch Normalization

- 如何进行测试

利用BN训练好模型后，我们保留了每组mini-batch训练数据在网络中每一层的 μ_{batch} 与 σ_{batch}^2 。此时我们使用整个样本的统计量来对Test数据进行归一化，具体来说使用均值与方差的无偏估计⁺：

$$\mu_{test} = \mathbb{E}(\mu_{batch})$$

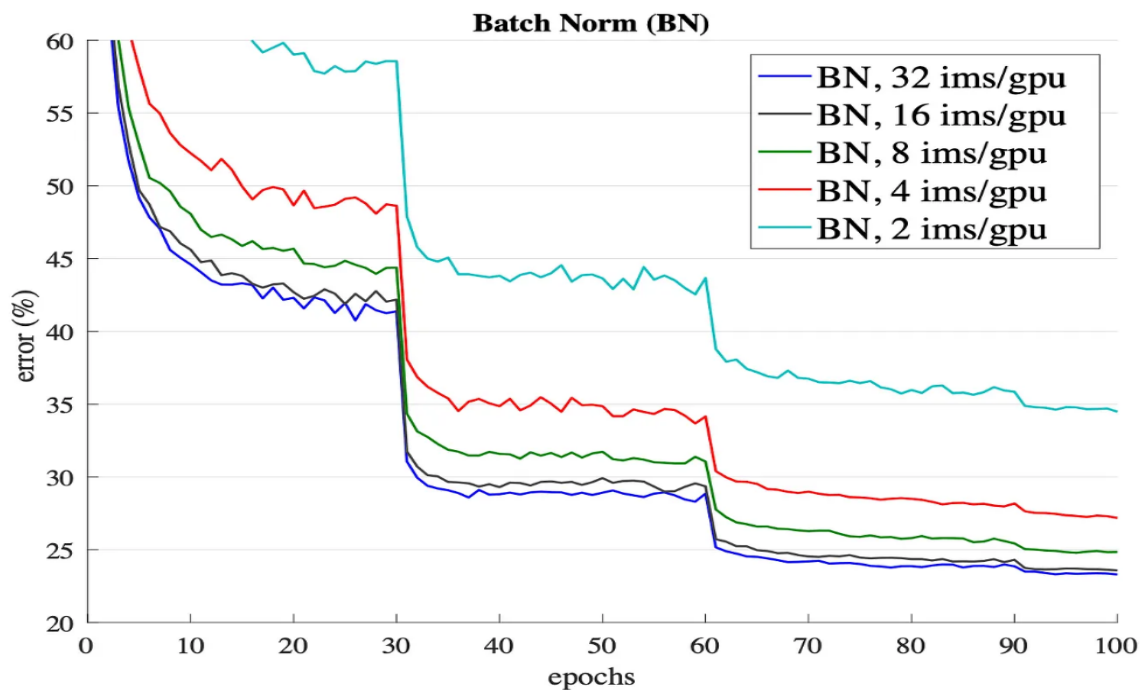
$$\sigma_{test}^2 = \frac{m}{m-1} \mathbb{E}(\sigma_{batch}^2)$$

得到每个特征的均值与方差的无偏估计后，我们对test数据采用同样的normalization方法：

$$BN(X_{test}) = \gamma \cdot \frac{X_{test} - \mu_{test}}{\sqrt{\sigma_{test}^2 + \epsilon}} + \beta$$

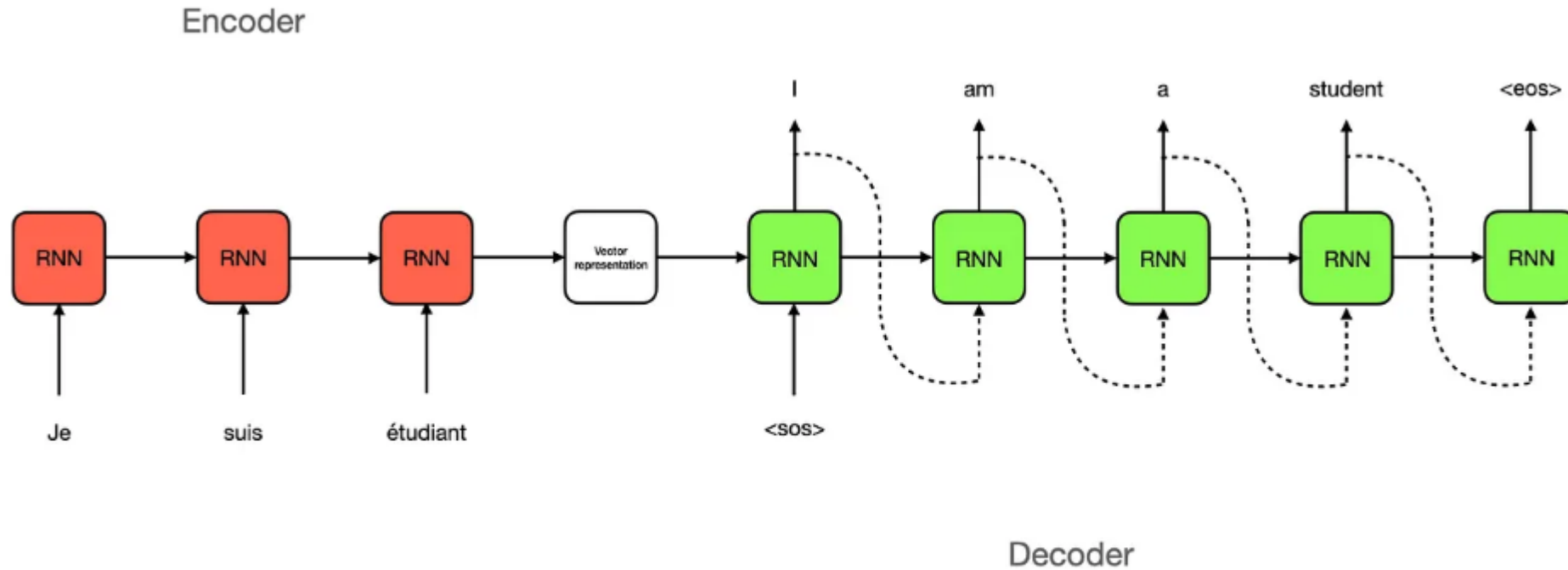
Batch Normalization

- 可以更容易训练深度网络
- 不易受到初始化参数的影响
- 由于分布固定，可以使用较大的学习率加快收敛速度。
- 但这是一把双刃剑。这种估计仅适用于较大的批次。当批中实例数量较少时，性能会急剧下降。



Layer Normalization

- Geoffrey E. Hinton 等人于 2016 年提出，用于解决BN性能受限于批量大小。另一个原因：循环神经网络应用 BN时收到限制。



Layer Normalization

- 层归一化计算每个单独实例的平均值和标准差（对于图像，则计算所有通道的平均值和标准差；对于 MLP，则计算所有特征的平均值和标准差），而不是整个批量。这使得它与批量大小无关，因此可用于 RNN 等模型或 Transformer 模型中。

Layer normalization 的计算可以分为两步：

1. **计算均值和方差**：对于给定的输入 $x \in \mathbb{R}^{B \times D}$ ，其中 $B(batchSize)$ 是批次大小， D 是特征维度，可以计算每个样本的均值和方差：

$$\text{均值: } \mu = \frac{1}{D} \sum_{i=1}^D x_i$$

$$\text{方差: } \sigma^2 = \frac{1}{D} \sum_{i=1}^D (x_i - \mu)^2$$

2. **标准化和重新缩放**：

$$\text{标准化后的输出: } \tilde{x}_i = \frac{x_i - \mu}{\sqrt{\sigma^2 + \epsilon}}$$

$$\text{重新缩放和偏移后的输出: } y_i = \gamma \tilde{x} + \beta$$

其中， ϵ 是一个很小的正数，用来防止除以零； γ 和 β 是可学习的参数，用于重新缩放和偏移。